LETTER *Special Section on Intelligent Information Processing Technology to be Integrated into Society*

Selecting Source Code Generation Tools Based on Bandit Algorithms

Masateru TSUNODA[†a)], ***Member***, **Ryoto SHIMA**[†], **Amjed TAHIR**[††], **Kwabena Ebo BENNIN**[†††], ***Nonmembers***, **Akito MONDEN**[††††], **Koji TODA**[†††††], ***Members***, ***and*** **Keitaro NAKASAI**[††††††], ***Nonmember***

SUMMARY Background: Code s generation tools such as GitHub Copilot have received attention due to their performance in generating code. Generally, a prior analysis of their performance is needed to select new code-generation tools from a list of candidates. Without such analysis, there is a higher risk of selecting an ineffective tool, which would negatively affect software development productivity. Additionally, conducting prior analysis of new code generation tools is often time-consuming. Aim: To use a new code generation tool without prior analysis but with low risk, we propose to evaluate the new tools during software development (i.e., online optimization). Method: We apply the bandit algorithm (BA) approach to help select the best code suggestion or generation tool among a list of candidates. Developers evaluate whether the result of the tool is correct or not. When code generation and evaluation are repeated, the evaluation results are saved. We utilize the stored evaluation results to select the best tool based on the BA approach. In our preliminary analysis, we evaluated five tools with 164 code-generation cases using BA. Result: BA approach selected ChatGPT as the best tool as the evaluation proceeded, and during the evaluation, the average accuracy by BA approach outperformed the second-best performing tool. Our results reveal the feasibility and effectiveness of BA in assisting the selection of best-performing code suggestion or generation tools.

key words: *software development, online optimization, multi-armed bandit problems, external validity*

1. Introduction

Recently, code suggestion or generation tools such as GitHub Copilot has attracted the attention of software developers [9] with their performance. The performance of the tools has been improved notably, and the tools are expected to improve productivity of software development. There are some suggestion or generation tools such as Amazon CodeWhisperer and GitHub Copilot, and their performances are considered to be different. To help developers to select a tool, Yetiştiren et al. [9] analyzed the performances as the correctness of several tools based on the test results of the generated code. Subsequently, the analysis result could be utilized as the guideline of the selection.

Assume that a new code generation tool α is released. A developer has used a tool β, and the new tool α has the potential to increase the productivity of the development team than β. However, the tool α might also have the risk of decreasing the productivity of the development team than β. To avoid the risk, analysis of the performance α and β is needed before using the tool. Based on the analysis, developers can decide whether the new tool α is a better option.

However, such analysis is often time-consuming and, therefore, challenging for developers to perform such analysis whenever a new tool is released. Additionally, the performance of the tools could be improved drastically within a short period, and therefore, the developers should often perform such analysis. For instance, the average correctness of generated codes by Amazon CodeWhisperer improved by 42% for about two months [9].

Instead of such analysis, we propose to evaluate code suggestion or generation tools during software development (i.e., online optimization) based on Bandit algorithm (BA). The major contributions are twofold. Firstly, we clarified the applicability of BA to the selection of code-generation tools. Finally, we evaluated the performance of BA for the selection.

2. Online Evaluation of Code Generation Tools Selection

2.1 Bandit Algorithms

Overview: BAs are proposed to solve multi-armed bandit problems. Those problems are often explained through an analogy with slot machines. Assume that a player has 100 coins to bet on several slot machines, and the player wants to maximize their reward. Instead of selecting only one slot machine and betting all 100 coins, BA suggests that a player bet only one coin on each slot machine. By calculating the average reward of each machine after each betting, the player can then recognize which slot machine is the best (i.e., the highest average reward). The derivation of the problem is that each slot machine has an arm, and the arm is compared to a bandit who steals money from players. Multi-armed bandit problem seeks sequentially best candidates (referred to as arms) whose expected rewards are unknown to maximize total rewards.

Manuscript received May 7, 2024.
Manuscript revised September 18, 2024.
Manuscript publicized November 11, 2024.

[†]Faculty of Informatics, Cyber Informatics Research Institute, Kindai University, Higashiosaka-shi, 577–8502 Japan.
[††]Massey University, New Zealand.
[†††]Wageningen University & Research, Netherlands.
[††††]Okayama University, Okayama-shi, 700–8530 Japan.
[†††††]Fukuoka Institute of Technology, Fukuoka-shi, 811–0295 Japan.
[††††††]Osaka Metropolitan University College of Technology, Neyagawa-shi, 572–0017 Japan.
a) E-mail: tsunoda@info.kindai.ac.jp

DOI: 10.1587/transinf.2024IIL0001

Copyright © 2025 The Institute of Electronics, Information and Communication Engineers

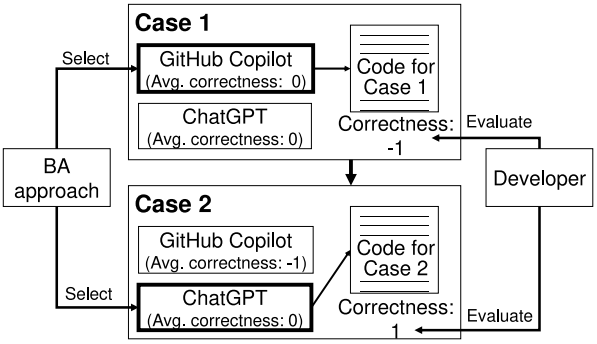

Fig. 1 Procedure of tool selection based on BA approach

BA method: The epsilon-greedy algorithm is a simple BA method. It chooses a random arm with probability epsilon. It selects an arm whose average reward is the highest among arms with the probability $1 - \varepsilon$ ($0 \leq \varepsilon \leq 1$). When the value of ε is 0, arms are always selected based on the average reward of each arm. In contrast, when the value of ε is 1, arms are always selected randomly.

Related work: In the domain of software engineering, BAs have been utilized to select methods such as feature selection methods for defect prediction [8] and configuration of clone detection methods [7]. In the other domains, BAs also have been used for compiler optimization [6], hyperparameter tuning [4], and performance tuning [5]. The aim of those studies [4]–[6] is to reduce the optimization time while also eliminating the need for human judgement. In contrast, our method weaves developer (i.e., human) judgement into coding activity to select the best method. The process can be regarded as a human-in-the-loop process. To the best of our knowledge, this is the first study to apply BAs to select code-generation tools. The definition of the reward is different from that of past studies, and therefore, it is not clear whether BAs will work well in selecting code-generation tools.

2.2 Proposed Approach

As an analysis approach, we propose to evaluate code suggestions or generation tools during coding. The procedure of our approach based on BA is the following:

1. A tool is selected randomly.
2. The developer inputs a prompt for the tool.
3. The developer examines the correctness of the tool's result (and utilizes the result).
4. The average correctness of each tool is calculated.
5. If the average correctness of the selected tool is low, another tool is selected.
6. Back to Step 2.

Step 3 is indispensable to use suggestion or generation tools, regardless of our approach, because there is no guarantee that the generated code is correct, even if the best performance tool is used. For instance, ChatGPT-4 has 62% of the generated code that contains API misuses [10]. Based on BA, our approach adds Steps 4 and 5 (i.e., evaluation and selection). Intuitively, our approach considers different generation tools as arms, and developers' evaluation of each tool is the reward.

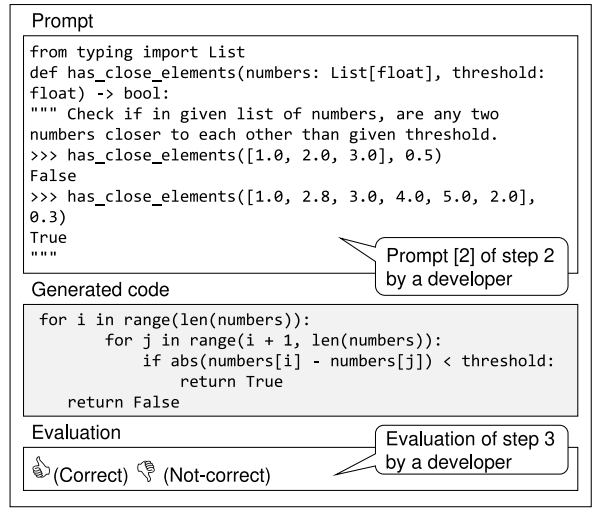

Fig. 2 Example of usage of the proposed approach on IDE

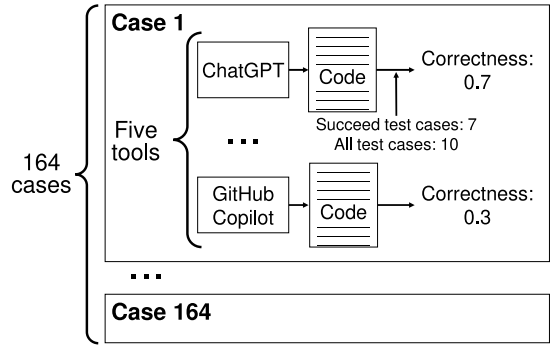

Fig. 3 Dataset used in the analysis

We show a concrete example of the behavior of our approach, assuming that our approach is implemented on IDE (integrated development environment), as shown in Fig. 2. In the figure, the prompt is a docstring comment of Python made by a study [2]. Initially, the average correctness of all tools is set to zero. Hence, for the first case in Fig. 1, GitHub Copilot is selected randomly in step 1. In step 3, a developer evaluates if the generated code is correct, and the correctness is then set to -1, since the developer regards it as "incorrect." Based on the evaluation, we calculate the average correctness for each tool. ChatGPT is selected in step 5, because the average correctness of ChatGPT is higher than GitHub Copilot. For the second case, in step 3, the developer regards the generated code as "correct." In step 5, based on the evaluation, the correctness is set to 1 since the developer regards it as "correct."

3. Experiment

Dataset: To evaluate the effectiveness of our approach, we performed a preliminary analysis. In the analysis, we used

Table 1 Average correctness of each tool [9]

Tool	GitHub Copilot Ver.1.7.4421	GitHub Copilot Ver.1.70.8099	Amazon CodeWhisperer Ver.Nov.22	Amazon CodeWhisperer Ver.Jan.23	ChatGPT
Average correctness	0.54	0.60	0.37	0.52	0.78

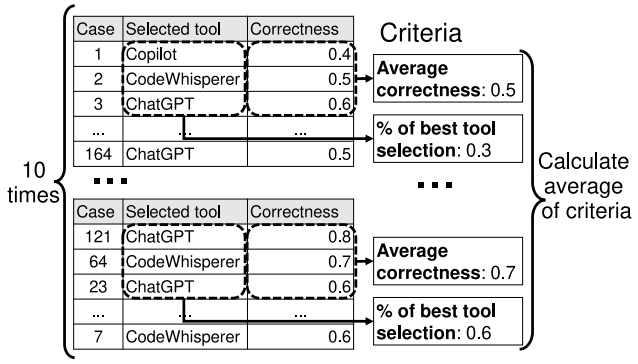

Fig. 4 Calculation of evaluation criteria

the dataset published by Yetiştiren et al. [9]. As shown in Fig. 3, the dataset includes 164 cases. For each case, the correctness of code generation tools is evaluated by the number of successful test cases divided by the number of all test cases. The value range of the correctness is $[0, 1]$, and a larger value denotes higher correctness.

Note that Yetiştiren et al. automatically calculated the correctness by using test cases provided by the other dataset [2]. However, such test cases are generally not provided before using code generation tool, and therefore correctness would be manually evaluated by developers (see Step 3 in Sect. 2.2).

Code generation tool: The dataset includes the correctness of five code suggestion or generation tools: GitHub Copilot (Version 1.7.4421 and 1.70.8099), Amazon CodeWhisperer (Version Nov.22 and Jan.23), and ChatGPT 3.5. In the analysis, BA tried to identify the best tool from the five tool candidates. Table 1 shows the average correctness of each tool in the 164 cases. As shown in the table, ChatGPT attained the highest correctness value.

Evaluation criteria: As an evaluation criterion, we used the average correctness and the percentage of best tool selection. As shown in Fig. 4, the average correctness of BA is based on the history of selected tools by BA.

For the analysis, 164 iterations of BA were performed. We analyzed the frequencies of the selected best tool (i.e., ChatGPT), picking up successive 10 iterations within the 164 iterations. We call the number of the selection "the percentage of the best tool selection". For instance, as shown in Fig. 4, when ChatGPT is selected once from the first to third iterations, we regarded the percentage as 33% at the third iteration (calculated based on the moving average).

The order of the correctness evaluation could affect BA's performance. Therefore, we randomly sorted 164 cases and evaluated BA's performance 10 times. After that, we calculated the average of the evaluation criteria, as shown in Fig. 4.

Table 2 Average correctness of proposed approach

Method	random (baseline)	$\varepsilon = 0.1$	$\varepsilon = 0.2$	$\varepsilon = 0.3$
Average correctness	0.56	0.63	0.65	0.68

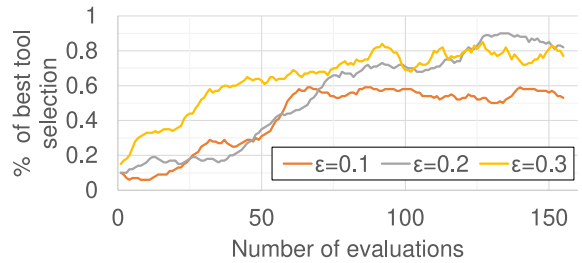

Fig. 5 Relationship between the number of evaluations and the percentage of best tool selection

Bandit Algorithm: For our BA setup, we used ε-greedy (ε = 0.1, 0.2 and 0.3). The performance of BA is affected by the order of cases. Therefore, we randomly sorted the cases and calculated the average of the evaluation criteria, as shown in Fig. 4.

In the analysis, we set the following research questions:

- **RQ1**. How effective is our proposed approach?
- **RQ2**. How often should developers evaluate the candidate tools to identify the best one?

For instance, when BA selects a tool twice with a correctness value of 0.4 and a tool whose correctness is 0.8 twice, the average correctness is 0.6. RQ1 is set to consider BA's correctness, and we calculated the average correctness when the tools are selected randomly for the baseline. To answer RQ2, we used the percentage of the best tool selection explained above.

Result: Table 2 shows the average correctness of the proposed approach and the baseline. As shown in the table, our approach achieved values higher than the baseline, and the average correctness was the highest when $\varepsilon = 0.3$. Additionally, compared with Table 1, the average correctness value of our approach was still higher than the second one (i.e., GitHub Copilot Ver.1.70.8099). Therefore, the answer to RQ1 is that our approach attains the second-best performance.

Figure 5 shows the relationship between the number of evaluations and the percentage of the best tool selection. When the number of evaluations was more than 75, the percentage of the best tool selection was higher than 0.7 in most cases. Therefore, the answer to RQ2 is that 75 evaluations are needed to select the best tool with a high probability.

In Fig. 5, when $\varepsilon = 0.1$, the percentage of the percentage

of best tool selection was smaller than 60%, even when the number of evaluations was 150. This might be because the exploration side of the algorithm was set too high.

Discussion: Based on the dataset used in the experiment, ChatGPT performed the highest. However, using ChatGPT is not always best compared with fine-tuned models. For instance, fine-tuned models produced better performances than ChatGPT on natural language processing tasks [1]. Additionally, as explained in Sect. 2.2, 62% of generated codes by ChatGPT contained API misuses [10]. The results suggest the necessity of our approach.

Prompt engineering is important to enhance the performance of large language models such as ChatGPT [3]. However, the program specifications shown in Fig. 2 are sufficient to generate source code, and therefore, we assume that prompt engineering is not very important for the task.

Although almost 75 iterations (i.e., evaluations) are needed to select the best tool, the evaluation is needed even if our approach is not applied, as explained in Sect. 2.2. Note that when we collect the evaluations concurrently from several developers through IDE (see Fig. 2), the iterations per developer will be shorten. Additionally, if the evaluation does not depend on the applied domain, we can utilize the evaluations collected in other development projects.

4. Conclusion

Instead of analyzing the performance of code suggestion or generation tools before applying them, we propose a new approach to selecting the best code generation tools during software development. The proposed approach is based on the Bandit algorithm. In the preliminary analysis, we tried to select the best tool from five candidates of suggestion or generation tools, such as GitHub Copilot, using the proposed approach. During the evaluation, our approach based on BA achieved the second-best performance, even when BA was not considered for tool selection and without prior analysis.

Acknowledgments

This research is partially supported by the Japan Society for the Promotion of Science [Grants-in-Aid for Scientific Research (C) (No.21K11840, 24K14896, 20H05706)].

References

[1] Y. Bang, S. Cahyawijaya, N. Lee, W. Dai, D. Su, B. Wilie, H. Lovenia, Z. Ji, T. Yu, W. Chung, Q.V. Do, Y. Xu, and P. Fung, "A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity," Proc. of International Joint Conference on Natural Language Processing and Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, vol.1, pp.675–718, 2023.

[2] M. Chen, J. Tworek, and H. Jun et al., "Evaluating Large Language Models Trained on Code," arXiv, 2107.03374, 2021.

[3] T. Kojima, S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large Language Models are Zero-Shot Reasoners," Advances in Neural Information Processing Systems, vol.35, pp.22199–22213, Curran Associates, Inc., 2022.

[4] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalka, "Hyperband: a novel bandit-based approach to hyperparameter optimization," Journal of Machine Learning Research, vol.18, no.1, pp.6765–6816, 2017.

[5] F. Mesmay, A. Rimmel, Y. Voronenko, and M. Püschel, "Bandit-based optimization on graphs with application to library performance tuning," Proc. of Annual International Conference on Machine Learning (ICML), pp.729–736, 2009.

[6] S. Park, S. Latifi, Y. Park, A. Behroozi, B. Jeon, and S. Mahlke, "SRTuner: Effective Compiler Optimization Customization by Exposing Synergistic Relations," International Symposium on Code Generation and Optimization (CGO), pp.118–130, 2022.

[7] T. Tabo, T. Kudo, M. Tsunoda, A. Tahir, K. Bennin, K. Toda, K. Nakasai, and A. Monden, "Clone Detection Method Selection Based on Bandit Algorithm: a Preliminary Analysis," Proc. of International Workshop on Software Clones (IWSC), pp.53–54, 2022.

[8] M. Tsunoda, A. Monden, K. Toda, A. Tahir, K.E. Bennin, K. Nakasai, M. Nagura, and K. Matsumoto, "Using Bandit Algorithms for Selecting Feature Reduction Techniques in Software Defect Prediction," Proc. of Mining Software Repositories Conference (MSR), pp.670–681, 2022.

[9] B. Yetiştiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT," arXiv, 2304.10778, 2023.

[10] L, Zhong and Z. Wang, "Can LLM Replace Stack Overflow? A Study on Robustness and Reliability of Large Language Model Code Generation," Proc. AAAI Conference on Artificial Intelligence, vol.38, no.19, pp.21841–21849, 2024.